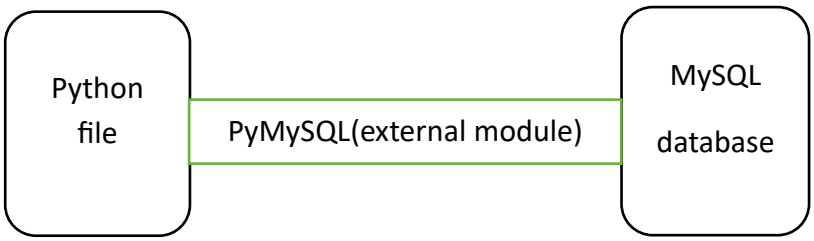
PYTHON DATABASE CONNECTION



>Data entered in Python file will be stored in the database table of MySQL through the PyMySQL module.

>It helps in serialization and deserialization of the data.

Step 1: Install pymysql Module

```
pip install pymysql
```

This installs the module that helps Python communicate with MySQL.

Step 2: **Import the Module**

```
import pymysql
```

This makes pymysql functions available in Python.

Step 3: Establish Connection

Creates a connection object

```
connection = pymysql.connect(
    host="localhost",
    user="root",
    password="your_password",
    database="your_database" )
```

Parameters:

host → MySQL server location (usually localhost)

user → MySQL username

password → MySQL password

database → Name of database to connect

- This creates a connection between Python and MySQL.
- Here Python sends request to MySQL server.
- MySQL verifies username & password.
- If correct → connection object is created.
- If wrong → connection error occurs.

Step 4: Create Cursor Object

```
cur = connection.cursor()
```

- Cursor object is used to execute SQL queries and interact with the database(that is it helps us to go inside the database and creating a table, creating the columns ,inserting the values, deleting , updating, retrieving the records).
- It acts as a bridge between Python and MySQL.
- Without cursor, we cannot run SQL commands.

Step 5: Performing basic CRUD operations

>reading the data:

```
cur.execute( “  sql query  “)
```

>creating a new database:

```
cur.execute(“create database databasename”)
```

>create a new table inside the database

```
cur.execute("create table tablename (columnname1 datatype,
                                     columnname2 datatype,
                                     ……,
                                     ……,
                                      columnname n datatype )")
```

>inserting the records:

```
cur.execute("insert into tablename
            (columname1,coloumname2…columnname n)
             values(val1,val2,…..val n)")
```

>inserting multiple records:

```
var1 = "insert into tablename (column1,column2,…column n)
             values(%s,%s,…,%s)")
var2 = [
      (val1,val2…) ,  (val1,val2…) , ……..,
      ]
```

var1 is to write the query

var2 is to provide values

```
cur.executemany(query,values)
```

Note:

- When we are inserting multiple values we need use executemany() method.
- we have to use lowercase s for placeholder.
- While writing the query we need to pass placeholders(%s) in the place of values.
- During the execution time we need to pass the values in the form of a collection(tuples inside a list) to the placeholders.
- At the execution time, it loops through each tuple and inserts them into the respective columns.

>updating the records:

cur.execute("sql query")

>deleting the records:

cur.execute("sql query")

Note:

We use **commit( )** method to save the changes permanently in the database.

When we perform operations like:

- Insert
- Update
- Delete

The changes are not save automatically, hence we must call commit( )

Step 6: closing the cursor object

cur.close()

>Once it is closed we cannot execute any SQL queries.
>But the connection object still remains active.

Step 7: closing the connection object

connection.close( )

>Once it is closed connection between python and the database is terminated.

Note:

1. Always close the cursor first, then close the connection .
2. Closing the cursor and connection object
    - Prevents resource leakage
    - Prevents errors
    - Improves program efficiency
    - Prevents too many open connections
    - Terminates database session properly
3. Closing is considered as good programming practice.

Simple Flow Diagram

- Install module
- Import module
- Create connection object
- Create cursor object
- Execute query
- Commit
- Close cursor object
- Close connection object
